

SLOT 12: Multer

Tóm tắt

28 User Authentication With Passport

- Use Passport Module Together With Passport-Local

29.1 use passport-local-mongoose for setting up local authentication within server

29.2 use passport together with sessions to set up user authentication

30 Token Based Authentication

 Bài học: Sử dụng Multer để Upload và Download File trong Express

◆ Chủ đề 34: Setup Express server to use Multer to upload files to a designated folder

Tóm tắt khái niệm:

Tìm hiểu cách cài đặt và cấu hình thư viện **Multer**, một middleware phổ biến trong Express, để hỗ trợ người dùng tải lên (upload) file từ client đến server.

Mục tiêu:

- Cài đặt và tích hợp Multer vào ứng dụng Express.
- Thiết lập thư mục lưu trữ file upload trên máy chủ.
- Xử lý request POST chứa dữ liệu dạng `multipart/form-data`.
- Hiểu cơ chế hoạt động của middleware Multer khi nhận file.

Sinh viên sẽ biết cách xây dựng endpoint nhận file từ client và lưu trữ chúng theo đúng cấu trúc mong muốn.

◆ Chủ đề 35: Configure the module to enable file downloading

Tóm tắt khái niệm:

Tìm hiểu cách thiết lập các route và logic để cho phép client tải xuống (download) các file đã được lưu trữ trên server.

Mục tiêu:

- Tạo endpoint cho phép truy xuất file dựa trên tên hoặc ID.
- Sử dụng `res.download()` hoặc `fs.createReadStream()` để gửi file về client.
- Áp dụng logic kiểm tra tính hợp lệ trước khi download (nếu cần).

Sinh viên sẽ biết cách tạo chức năng download file từ server, đảm bảo an toàn và hiệu quả.

◆ Chủ đề 36: Setup Express server to use Multer to download files to a designated folder

Tóm tắt khái niệm:

Mặc dù Multer chủ yếu dùng để upload, nhưng phần này sẽ hướng dẫn sinh viên kết hợp Multer với các kỹ thuật khác để hoàn thiện hệ thống quản lý file bao gồm cả upload và download.

Mục tiêu:

- Tích hợp Multer với các module như `fs`, `path` để quản lý file.
- Tự động đồng bộ danh sách file đã upload.
- Cho phép người dùng chọn file để tải về từ danh sách có sẵn.

Sinh viên sẽ xây dựng được hệ thống quản lý file hai chiều: *upload & download*.

Tài liệu tham khảo

1. **Multer Documentation – GitHub** [multer/doc/README-vi.md at main · expressjs/multer](https://github.com/expressjs/multer/blob/main/README-vi.md)
Tài liệu chính thức của Multer, mô tả đầy đủ các tùy chọn cấu hình, storage engine và ví dụ thực tế.
2. **Express.js Official Guide – expressjs.com** <https://expressjs.com/>
Hướng dẫn sử dụng Express.js, bao gồm các middleware phổ biến như Multer, body-parser, v.v.
3. **"Node.js Design Patterns" – Mario Casciaro**
Cuốn sách chuyên sâu về kiến trúc backend với Node.js, bao gồm các pattern xử lý file, stream, upload/download.
4. **freeCodeCamp – How to Upload Files in Node.js with Multer** <https://www.freecodecamp.org/news/how-to-upload-files-in-node-js-with-multer/>
Bài viết hướng dẫn từng bước chi tiết đi kèm ví dụ thực tế, dễ hiểu cho người mới bắt đầu.
5. **Traversy Media – YouTube (ví dụ: "File Uploads with Node.js & Express Tutorial")**
Video hướng dẫn trực quan, minh họa cách upload và download file bằng Express + Multer.

CÁC NỘI DUNG TRỌNG TÂM

 Chủ đề 34: Setup Express server to use Multer to upload files to a designated folder

Khái niệm quan trọng:

- `multipart/form-data`: Định dạng dữ liệu dùng để tải file lên từ form HTML hoặc client.
- Middleware Multer: Thư viện hỗ trợ xử lý upload file trong Express.
- Storage Engine: Cơ chế lưu trữ file (`diskStorage`, `memoryStorage`...).
- `req.file` / `req.files`: Đối tượng chứa thông tin file sau khi upload.

Kỹ thuật cần thành thạo:

- Cài đặt và cấu hình Multer trong ứng dụng Express.
- Tạo thư mục lưu trữ file upload trên hệ thống.
- Sử dụng `multer({ storage })` để tùy chỉnh đường dẫn và tên file.
- Thiết lập route POST nhận file upload với middleware Multer.
- Xử lý upload nhiều file (`array()` hoặc `fields()`).

Lỗi thường gặp:

- Không gửi đúng định dạng `multipart/form-data` từ client → không nhận được file.

- Thiếu middleware `upload.single()` hoặc `upload.array()` → lỗi 400 hoặc không parse được file.
- Quên tạo thư mục đích trước → lỗi permission hoặc không lưu file.
- Trùng tên file → ghi đè hoặc lỗi logic.

✅ Best Practices:

- Luôn kiểm tra loại file và kích thước trước khi cho phép upload.
- Đặt tên file ngẫu nhiên hoặc có tính duy nhất để tránh xung đột.
- Hạn chế dung lượng upload bằng `.limits()` để tránh overload server.
- Phân tách cấu hình Multer ra một file riêng để dễ bảo trì.

Sinh viên cần hiểu rõ cách hoạt động của Multer và thành thạo việc setup upload file đơn giản đến nâng cao.

📦 Chủ đề 35: Configure the module to enable file downloading

✅ Khái niệm quan trọng:

- `res.download()`: Gửi file về client như một file đính kèm để tải xuống.
- `fs.createReadStream() + res.pipe()`: Cách thay thế linh hoạt hơn để stream file.
- Quản lý quyền truy cập file (authorization) trước khi download.

✂ Kỹ thuật cần thành thạo:

- Tạo endpoint nhận tham số tên file hoặc ID file.
- Kiểm tra sự tồn tại của file trước khi download.
- Dùng `res.download()` để trả file trực tiếp.
- Hiển thị danh sách file đã upload (nếu cần).
- Kết hợp với `path.join()` để truy xuất file an toàn.

⚠ Lỗi thường gặp:

- Đường dẫn file sai → lỗi 404 hoặc không tìm thấy file.
- Cho phép tải file ngoài thư mục được chỉ định → rủi ro bảo mật.
- Không bắt lỗi đọc file → crash server nếu file bị xóa hoặc hỏng.

✅ Best Practices:

- Luôn validate đầu vào để đảm bảo người dùng chỉ tải file hợp lệ.
- Không expose toàn bộ cấu trúc thư mục hệ thống.
- Log lại các hành vi download nếu cần thiết để audit.

Sinh viên cần biết cách xây dựng chức năng download file một cách an toàn và hiệu quả.

◆ Chủ đề 36: Setup Express server to use Multer to download files to a designated folder

✓ Khái niệm quan trọng:

- Đồng bộ hóa giữa upload và download: Sau khi upload, file phải có thể truy xuất để download.
- Quản lý trạng thái file (tên, đường dẫn, thời gian upload...).
- Tích hợp Multer với `fs`, `path`, `express.static()` nếu cần.

✂ Kỹ thuật cần thành thạo:

- Lưu trữ metadata file (tên, loại, đường dẫn...) vào DB hoặc biến tạm.
- Tự động đồng bộ danh sách file có sẵn.
- Tạo giao diện đơn giản hoặc API trả về danh sách file có thể download.
- Tùy chọn: sử dụng `express.static()` để phục vụ file tĩnh (public).

⚠ Lỗi thường gặp:

- Không đồng bộ giữa tên file lưu trữ và tên file download.
- Không xử lý trường hợp file bị xoá hoặc trùng tên.
- Không phân quyền ai được download file nào.

✓ Best Practices:

- Dùng thư mục chung để quản lý cả upload và download.
- Tạo API trả về danh sách file hiện có (GET `/files`) để tiện tích hợp frontend.
- Bảo vệ thư mục upload khỏi truy cập công khai nếu chưa muốn public file.

Sinh viên sẽ kết nối được quy trình đầy đủ từ upload -> lưu trữ -> hiển thị -> download file.

🔗 Mục tiêu cuối cùng của bài học:

Sau bài học này, sinh viên **bắt buộc phải** :

- Thành thạo cấu hình Multer để upload file đơn và nhiều file.
- Biết cách thiết lập endpoint download file an toàn.
- Có khả năng tự xây dựng một module nhỏ quản lý file upload/download.
- Hiểu rõ các best practices và lỗi phổ biến để tránh trong thực tế

ỨNG DỤNG THỰC TIỄN TRONG BÀI HỌC:

"Xử lý file với Multer – Upload và Download trong ứng dụng thực tế"

◆ **TÌNH HUỐNG 1:** Hệ thống nộp bài tập trực tuyến (Online Assignment Submission System)

Bối cảnh:

Một trường đại học triển khai nền tảng học tập trực tuyến, nơi sinh viên có thể tải lên bài tập (file PDF, Word, Excel...) và giảng viên có thể tải xuống để chấm điểm.

Vấn đề:

- Cần xử lý upload nhiều loại file từ hàng ngàn sinh viên mỗi ngày.
- Giảng viên cần truy xuất từng bài tập dễ dàng.
- Đảm bảo tính an toàn, tránh xung đột tên file, quản lý thư mục hợp lý.

Giải pháp – Áp dụng kỹ thuật từ bài học:

34. Setup Multer để upload file	Cấu hình <code>multer.diskStorage</code> để lưu file vào thư mục như <code>/uploads/assignment/</code> . Tên file được đặt theo định dạng <code>{studentId}_{timestamp}_{originalName}</code> để tránh trùng lặp.	Cho phép sinh viên tải lên bài tập một cách tự động và không bị ghi đè file.
35. Configure download module	Tạo route <code>/download/:filename</code> , sử dụng <code>res.download()</code> hoặc <code>fs.createReadStream()</code> để trả file về client. Kết hợp với middleware kiểm tra quyền truy cập.	Giảng viên chỉ tải xuống những file họ có quyền xem.
36. Setup Multer kết hợp download	Xây dựng API <code>/assignments/list</code> trả về danh sách file đã upload dựa trên khóa học. Giao diện frontend hiển thị danh sách bài tập kèm nút download.	Tích hợp quy trình đầy đủ từ upload -> quản lý -> download.

Kết quả đạt được:

- Sinh viên nộp bài dễ dàng, hệ thống xử lý ổn định ngay cả khi có lượng lớn file.
- Giảng viên truy xuất file chính xác, nhanh chóng.
- Quản lý file tập trung, giảm thiểu rủi ro bảo mật.

◆ **TÌNH HUỐNG 2:** Nền tảng chia sẻ tài liệu nội bộ công ty (Internal Document Sharing Platform)

Bối cảnh:

Một công ty startup xây dựng hệ thống nội bộ cho nhân viên upload tài liệu hướng dẫn, báo cáo, hợp đồng... và download khi cần thiết.

Vấn đề:

- Tài liệu nhạy cảm cần được kiểm soát chặt chẽ.
- Nhiều nhân viên cùng upload/download trong thời gian ngắn → cần hiệu suất cao.
- Không muốn dùng cloud storage vì lý do chi phí và kiểm soát dữ liệu.

Giải pháp – Áp dụng kỹ thuật từ bài học:

34. Setup Multer để upload file	Sử dụng Multer để nhận file upload từ form hoặc API, cấu hình đường dẫn theo phòng ban: <code>/uploads/hr</code> , <code>/uploads/marketing</code> , v.v.	Phân loại tài liệu theo nhóm, để quản lý.
35. Configure download module	Mỗi tài liệu có ID riêng, route <code>/documents/:id/download</code> sẽ tìm file tương ứng và gửi về người dùng sau khi xác thực.	Chỉ nhân viên có quyền mới được tải tài liệu.
36. Setup Multer kết hợp download	Tích hợp với database để lưu metadata (tên file, loại, người upload, thời gian), từ đó phục vụ list view và chức năng tìm kiếm.	Đồng bộ hóa quy trình upload - lưu trữ - hiển thị - download.

Kết quả đạt được:

- Tài liệu được phân loại rõ ràng theo phòng ban.
- Hệ thống hoạt động ổn định ngay cả khi có nhiều người dùng.
- Quyền truy cập được kiểm soát chặt chẽ, đảm bảo an toàn thông tin.

CASE STUDY THỰC TẾ: Dự án "EduShare" – Một startup giáo dục tại Việt Nam

Giới thiệu:

EduShare là nền tảng học tập trực tuyến giúp giáo viên tạo lớp học, giao bài tập và sinh viên nộp bài dưới dạng file. Startup này gặp phải vấn đề về xử lý file khi lượng người dùng tăng nhanh.

Vấn đề:

- Hệ thống cũ dùng base64 để truyền file, gây chậm và nặng server.
- File bị ghi đè nếu trùng tên.
- Không hỗ trợ download nhiều file cùng lúc.
- Thiếu kiểm soát quyền truy cập file.

Giải pháp:

Họ chuyển sang sử dụng **Express + Multer** để xử lý upload và download file như sau:

- **Upload file:**
Dùng `multer.diskStorage` để lưu file vào `/uploads/assignments/`, tên file được mã hóa duy nhất.
Kiểm tra loại file (PDF, DOCX, XLSX) trước khi chấp nhận upload.
- **Download file:**
Tạo route `/api/files/:id/download` kết hợp với xác thực JWT.
Dùng `res.download(filePath)` để trả file về client.

- **Quản lý file:**

Lưu metadata file (tên, loại, người upload, đường dẫn) vào MongoDB.

Tạo giao diện quản lý file giúp giáo viên duyệt/xóa/tải xuống.



Kết quả:

- Hiệu suất cải thiện rõ rệt: tải file nhanh hơn 70%.
- Không còn lỗi ghi đè file nhờ tên file duy nhất.
- Giao diện quản lý file giúp giáo viên làm việc thuận tiện hơn.
- Chi phí vận hành giảm do không dùng cloud storage.



BÀI HỌC RÚT RA:

1. **Hiểu rõ từng giai đoạn trong quy trình xử lý file (upload → lưu trữ → download)** là yếu tố then chốt để xây dựng ứng dụng mạnh mẽ.
2. **Phân quyền hợp lý và kiểm tra đầu vào cẩn thận** là rất quan trọng để đảm bảo an toàn dữ liệu.
3. **Kết hợp Multer với các module như fs, path và database** sẽ giúp mở rộng khả năng quản lý file.
4. **Không nên lưu file trong memory nếu file lớn** – hãy dùng `diskStorage`.
5. **Luôn giới hạn kích thước file và kiểm tra định dạng** để tránh overload server.



MỤC TIÊU GIẢNG DẠY CHO SINH VIÊN QUA PHẦN NÀY:

Sau khi học xong, sinh viên sẽ:

- Hiểu rõ vai trò của Multer trong phát triển ứng dụng backend.
- Biết cách áp dụng kỹ thuật upload/download vào các dự án thực tế.
- Có tư duy giải quyết vấn đề liên quan đến quản lý file trong môi trường sản phẩm thật.
- Chuẩn bị tốt để tiếp cận các dự án lớn hơn như tích hợp cloud storage (AWS S3, Google Cloud Storage), stream video/audio...

CHI TIẾT MỘT SỐ NỘI DUNG

KỸ THUẬT 34: Setup Express server to use Multer to upload files to a designated folder

Khái niệm và vai trò

Multer là một middleware phổ biến trong Express.js giúp xử lý dữ liệu dạng `multipart/form-data`, thường dùng để nhận file upload từ client.

Vai trò:

- Nhận file từ form hoặc API request.
- Lưu trữ file vào thư mục chỉ định trên server.
- Cung cấp thông tin chi tiết về file như tên, loại, kích thước, đường dẫn...

Tại sao quan trọng?

Trong backend, việc cho phép người dùng tải lên hình ảnh, tài liệu, video... là nhu cầu thiết yếu. Multer cung cấp giải pháp đơn giản, hiệu quả và dễ mở rộng cho chức năng này.

Các bước triển khai (Step-by-step)

Bước 1: Cài đặt Multer

```
bash
```

```
npm install multer
```

Bước 2: Tạo thư mục lưu trữ file

Tạo một thư mục như `/public/uploads` để chứa các file sau khi upload.

Bước 3: Cấu hình Multer

```
js
```

```
const express = require('express');
const multer = require('multer');
const path = require('path');

// Thiết lập nơi lưu trữ file
const storage = multer.diskStorage({
  destination: function (req, file, cb) {
    cb(null, 'public/uploads/'); // Thư mục lưu trữ
  },
  filename: function (req, file, cb) {
    const uniqueSuffix = Date.now() + '-' + Math.round(Math.random() * 1E9);
    cb(null, file.fieldname + '-' + uniqueSuffix +
      path.extname(file.originalname)); // Đặt tên duy nhất
  }
});
```



```
const upload = multer({ storage: storage });
```

Bước 4: Tạo route upload

js

```
const app = express();

app.post('/upload', upload.single('avatar'), (req, res) => {
  console.log(req.file); // Thông tin file đã upload
  res.status(200).json({
    message: 'Upload thành công',
    filename: req.file.filename
  });
});
```

Client gửi POST đến /upload với FormData có key là avatar.



Ví dụ minh họa đầy đủ

Server-side (server.js)

js

```
const express = require('express');
const multer = require('multer');
const path = require('path');
const app = express();
const PORT = 3000;

const storage = multer.diskStorage({
  destination: (req, file, cb) => {
    cb(null, 'public/uploads/');
  },
  filename: (req, file, cb) => {
    const uniqueName = `${Date.now()}-${Math.round(Math.random() * 1E9)}${path.extname(file.originalname)}`;
    cb(null, uniqueName);
  }
});

const upload = multer({ storage });

app.post('/upload', upload.single('file'), (req, res) => {
  if (!req.file) {
    return res.status(400).json({ error: 'Không có file nào được chọn' });
  }

  res.json({
    message: 'Upload thành công',
    filename: req.file.filename
  });
});

app.listen(PORT, () => {
  console.log(`Server đang chạy tại http://localhost:${PORT}`);
});
```

```
});
```

Client-side (HTML Form)

html

```
<form action="/upload" method="POST" enctype="multipart/form-data">
<input type="file" name="file">
<button type="submit">Upload</button>
</form>
```

Lỗi thường gặp và cách khắc phục

Không nhận được req. file	Thiếu <code>enctype="multipart/form-data"</code> ở form	Kiểm tra lại form HTML
File không được lưu	Thiếu thư mục đích hoặc quyền ghi	Tạo thư mục trước và kiểm tra quyền truy cập
Tên file bị trùng	Không tùy chỉnh <code>filename</code>	Dùng timestamp hoặc random string
Upload nhiều file thất bại	Dùng <code>.single()</code> thay vì <code>.array()</code>	Dùng <code>upload.array('field', limit)</code> nếu cần nhiều file

Best Practices

- Luôn kiểm tra loại file và giới hạn kích thước bằng `.limits()`
- Đặt tên file duy nhất để tránh xung đột
- Phân tách cấu hình Multer sang một file riêng (ví dụ: `multerConfig.js`)
- Hạn chế upload file quá lớn để tránh overload server

Kết hợp với các kỹ thuật khác

- **Kỹ thuật 35:** Sau khi upload, file có thể được download bằng `res.download()`.
- **Kỹ thuật 36:** Đồng bộ hóa giữa upload và download bằng cách lưu metadata file (tên, đường dẫn...) vào DB.
- Có thể kết hợp với **Cloud Storage** (AWS S3, Firebase Storage...) thay vì lưu local.

KỸ THUẬT 35: Configure the module to enable file downloading

Khái niệm và vai trò

Cho phép client tải xuống các file đã được lưu trữ trên server. Thường dùng trong các ứng dụng như hệ thống quản lý tài liệu, nền tảng chia sẻ file, blog có đính kèm tài nguyên...

Vai trò:

- Gửi file từ server về client dưới dạng download
- Kiểm soát quyền truy xuất file
- Tăng tính tương tác và tiện ích cho ứng dụng

Các bước triển khai (Step-by-step)

Bước 1: Tạo endpoint download

js

```
app.get('/download/:filename', (req, res) => {
  const filePath = path.join(__dirname, 'public/uploads/',
    req.params.filename);
  res.download(filePath); // Gửi file về client
});
```

Bước 2: Kiểm tra sự tồn tại của file (nâng cao)

js

```
const fs = require('fs');

app.get('/download/:filename', (req, res) => {
  const filePath = path.join(__dirname, 'public/uploads/',
    req.params.filename);

  if (!fs.existsSync(filePath)) {
    return res.status(404).json({ error: 'File không tồn tại' });
  }

  res.download(filePath);
});
```

Ví dụ đầy đủ

js

```
const express = require('express');
const path = require('path');
const fs = require('fs');
const app = express();

app.get('/download/:filename', (req, res) => {
  const filePath = path.join(__dirname, 'public/uploads/',
    req.params.filename);

  if (!fs.existsSync(filePath)) {
    return res.status(404).send('File không tồn tại');
  }

  res.download(filePath, (err) => {
    if (err) {
      console.error(err);
      res.status(500).send('Lỗi khi tải file');
    }
  });
});

app.listen(3000, () => {
  console.log('Server đang chạy tại http://localhost:3000');
});
```

Lỗi thường gặp

File không tìm thấy	Đường dẫn sai hoặc file chưa tồn tại	Kiểm tra <code>fs.existsSync()</code>
Truy xuất file ngoài thư mục	Sử dụng <code>..</code> trong URL	Dùng <code>path.normalize()</code> để đảm bảo an toàn

Crash server khi đọc file hỏng	Không bắt lỗi trong callback của <code>res.download()</code>	Luôn bọc trong try-catch hoặc dùng callback
---------------------------------------	--	---

✓ Best Practices

- Luôn kiểm tra sự tồn tại của file trước khi download
- Không expose toàn bộ cấu trúc thư mục hệ thống
- Log lại hành vi download để audit (nếu cần)
- Có thể thêm xác thực (JWT, session...) để phân quyền download

Kết hợp với các kỹ thuật

- **Kỹ thuật 34:** File phải được upload thành công trước khi có thể download
- **Kỹ thuật 36:** Cần đồng bộ giữa upload và download bằng cách lưu metadata file
- Có thể tích hợp với `express.static()` nếu muốn public một số file nhất định

KỸ THUẬT 36: Setup Express server to use Multer to download files to a designated folder

Khái niệm và vai trò

Mặc dù Multer chủ yếu hỗ trợ upload, nhưng nó có thể được kết hợp với các module khác như `fs`, `path` để xây dựng hệ thống quản lý file hai chiều: **upload** ↔ **download**

✓ Vai trò:

- Tích hợp quy trình upload và download
- Đồng bộ trạng thái file
- Xây dựng giao diện hoặc API trả về danh sách file có thể tải về

Các bước triển khai (Step-by-step)

Bước 1: Upload file và lưu metadata

Sau khi upload file, lưu tên file, loại, người upload vào database hoặc biến tạm.

Bước 2: Tạo API trả về danh sách file

js

```
app.get('/files', (req, res) => {
  fs.readdir(path.join(__dirname, 'public/uploads'), (err, files) => {
    if (err) return res.status(500).send(err);
    res.json(files);
  });
});
```

Bước 3: Cho phép tải file từ danh sách

js

```
app.get('/download/:filename', (req, res) => {
  const filePath = path.join(__dirname, 'public/uploads', req.params.filename);
  res.download(filePath);
});
```

Ví dụ đầy đủ

js

```
const express = require('express');
const multer = require('multer');
const fs = require('fs');
const path = require('path');

const app = express();

// Cấu hình Multer
const storage = multer.diskStorage({
  destination: './public/uploads/',
  filename: (req, file, cb) => {
    cb(null, Date.now() + '-' + file.originalname);
  }
});
const upload = multer({ storage });

// Upload
app.post('/upload', upload.single('file'), (req, res) => {
  res.json({ message: 'Upload thành công', filename: req.file.filename });
});
```

// Danh sách file

```
app.get('/files', (req, res) => {
  fs.readdir(path.join(__dirname, 'public/uploads'), (err, files) => {
    if (err) return res.status(500).send(err);
    res.json(files);
  });
});
```

// Download

```
app.get('/download/:filename', (req, res) => {
  const filePath = path.join(__dirname, 'public/uploads', req.params.filename);
  res.download(filePath);
});

app.listen(3000, () => {
  console.log('Server đang chạy tại http://localhost:3000');
});
```

Lỗi thường gặp

File upload không hiện trong danh sách	Chưa reload hoặc cache	Thêm logic refresh hoặc xóa cache
Không hiển thị file sau khi upload	Không update UI hoặc API	Dùng socket hoặc fetch lại danh sách sau upload
Không đồng bộ giữa tên file lưu trữ và tên download	Không kiểm soát tên file	Dùng logic tạo tên duy nhất và lưu metadata

✓ Best Practices

- Dùng database để lưu metadata file (tên, loại, người upload...)
- Hiển thị danh sách file qua API JSON
- Kiểm soát quyền truy cập từng file dựa trên user role
- Tự động xoá file cũ nếu không còn cần thiết

Kết hợp với các kỹ thuật

- **Kỹ thuật 34:** Upload file thành công trước khi có thể quản lý và download
- **Kỹ thuật 35:** Download file sau khi upload
- Có thể tích hợp với frontend framework như React để xây dựng UI quản lý file

Tổng kết liên kết giữa các kỹ thuật

34. Upload file	Nhận file từ client	35, 36
35. Download file	Gửi file về client	34, 36
36. Quản lý file	Đồng bộ upload/download	34, 35

KINH NGHIỆM THỰC TIỄN

 KỸ THUẬT 34: Setup Express server to use Multer to upload files to a designated folder

 KINH NGHIỆM THỰC TẾ & MẸO LẬP TRÌNH

 Vấn đề ngoài lý thuyết thường gặp:

- Không xử lý file trùng tên → ghi đè dữ liệu quan trọng
- Upload file quá lớn → crash server hoặc làm chậm ứng dụng
- Thiếu kiểm tra loại file → người dùng có thể upload bất kỳ định dạng nào (kể cả script nguy hiểm)
- Thiếu logic xác thực trước khi upload → ai cũng có thể tải file lên

 Tình huống thực tế:

Trong một hệ thống nộp bài tập trực tuyến, một sinh viên vô tình upload file tên `report.docx`. Một sinh viên khác sau đó cũng upload file cùng tên → file cũ bị ghi đè. Kết quả: giảng viên không tìm thấy bài tập của sinh viên đầu tiên.

 Giải pháp:

Sử dụng logic đặt tên file duy nhất:

js

```
filename: function (req, file, cb) {  
  const uniqueName = `${Date.now()}-${Math.round(Math.random() *  
  1E9)}${path.extname(file.originalname)}`;  
  cb(null, uniqueName);  
}
```

LỖI THƯỜNG GẶP

Không nhận được req.file	Thiếu <code>enctype="multipart/form-data"</code> ở form	Kiểm tra lại HTML form
Upload thành công nhưng không lưu file	Thư mục đích không tồn tại hoặc thiếu quyền ghi	Tạo thư mục trước và kiểm tra permission
File bị hỏng hoặc không đọc được	Upload không hoàn tất hoặc không bắt lỗi	Dùng callback để bắt lỗi từ <code>multer</code>
Upload nhiều file thất bại	Dùng <code>.single()</code> thay vì <code>.array()</code> hoặc <code>.fields()</code>	Chọn đúng middleware theo số lượng và loại file

MẸO LẬP TRÌNH

- **Kiểm tra loại file trước khi upload:**

```
js
const fileFilter = (req, file, cb) => {
  const allowedTypes = /jpeg|jpg|png|pdf/;
  const extname =
    allowedTypes.test(path.extname(file.originalname).toLowerCase());
  const mimetype = allowedTypes.test(file.mimetype);

  if (mimetype && extname) {
    return cb(null, true);
  }
  cb('Chỉ hỗ trợ hình ảnh và PDF');
};
```

- **Phân tách cấu hình Multer ra một file riêng (ví dụ: `upload.js`) để dễ bảo trì**
- **Log thông tin file upload để debug dễ dàng hơn**
- **Dùng `multer.memoryStorage()` nếu muốn xử lý file trong bộ nhớ (dành cho file nhỏ)**

BEST PRACTICES

- Luôn giới hạn kích thước file bằng `.limits({ fileSize: 1024 * 1024 * 5 })`
- Đặt tên file duy nhất để tránh xung đột
- Phân loại file theo thư mục (ví dụ: `/uploads/images/, /uploads/docs/`)
- Kiểm tra loại file và chặn các định dạng nguy hiểm
- Xác thực người dùng trước khi cho phép upload
- Bảo vệ thư mục uploads khỏi truy cập công khai (nếu chưa cần thiết)

KỸ THUẬT 35: Configure the module to enable file downloading

KINH NGHIỆM THỰC TẾ & MẸO LẬP TRÌNH

Vấn đề ngoài lý thuyết thường gặp:

- Cho phép tải file ngoài thư mục chỉ định (path traversal attack)
- Không kiểm soát quyền truy xuất → mọi người đều tải file nhạy cảm
- File đã bị xóa hoặc hỏng → gây lỗi 500 trên server

Tình huống thực tế:

Trong một nền tảng chia sẻ tài liệu nội bộ, một nhân viên sử dụng URL như `/download/../../config/passwords.txt` để tải file hệ thống → rò rỉ dữ liệu nhạy cảm.

Giải pháp:

Dùng `path.normalize()` để đảm bảo đường dẫn hợp lệ:

```
js
```



```
const filePath = path.join(__dirname, 'public/uploads', req.params.filename);
if (!filePath.startsWith(path.join(__dirname, 'public/uploads'))) {
  return res.status(403).send('Truy cập bị từ chối');
}
```

⚠ LỖI THƯỜNG GẶP

Không download được file	Đường dẫn sai hoặc file không tồn tại	Dùng <code>fs.existsSync(filePath)</code> để kiểm tra
Server crash khi đọc file hỏng	Không bắt lỗi trong callback	Dùng callback với <code>res.download(filePath, err => {...})</code>
Cho phép tải file ngoài thư mục	Dùng <code>..</code> trong URL	Dùng <code>path.normalize()</code> và kiểm tra đường dẫn gốc
Download chậm do stream không tối ưu	Không dùng stream phù hợp	Dùng <code>fs.createReadStream().pipe(res)</code> nếu cần điều chỉnh stream

💡 MẸO LẬP TRÌNH

- Luôn kiểm tra sự tồn tại của file trước khi download
- Dùng `res.header('Content-Type', 'application/octet-stream')` nếu muốn force download thay vì preview
- Ghi log mỗi lần download để audit (nếu là hệ thống nhạy cảm)
- Xác thực JWT hoặc session trước khi cho phép download file

🛡 BEST PRACTICES

- Chỉ cho phép download những file nằm trong thư mục chỉ định
- Có phân quyền rõ ràng (role-based access)
- Hạn chế số lần hoặc tốc độ download nếu cần
- Ghi log hành vi download
- Không expose toàn bộ danh sách file nếu không cần thiết

🔧 KỸ THUẬT 36: Setup Express server to use Multer to download files to a designated folder

✅ KINH NGHIỆM THỰC TẾ & MẸO LẬP TRÌNH

🔴 Vấn đề ngoài lý thuyết thường gặp:

- Không đồng bộ giữa upload và download → file upload xong nhưng không hiện trong danh sách
- Không quản lý metadata → khó tìm kiếm hoặc phân tích dữ liệu
- Không xử lý lỗi khi file bị xóa hoặc rename thủ công

📖 Tình huống thực tế:

Trong một ứng dụng quản lý hồ sơ bệnh án, bác sĩ upload file PDF chứa kết quả xét nghiệm, nhưng hệ thống không hiển thị file mới trong danh sách → phải reload nhiều lần mới thấy.

💡 Giải pháp:

Lưu metadata vào database mỗi khi upload:

js

```
app.post('/upload', upload.single('file'), (req, res) => {
  const fileData = {
    filename: req.file.filename,
    originalName: req.file.originalname,
    uploadedAt: new Date(),
    uploader: req.user.id
  };
  db.collection('files').insertOne(fileData);
  res.json({ message: 'Upload thành công' });
});
```

⚠ LỖI THƯỜNG GẶP

File upload xong nhưng không hiện trong list	Không update UI hoặc API trả về list cũ	Fetch lại danh sách sau upload
Không tìm thấy file dù đã upload	File bị xóa hoặc rename thủ công	Đồng bộ với DB và kiểm tra tồn tại trước khi download
Hiển thị danh sách file chậm	Quét thư mục bằng fs.readdir mỗi lần	Cache danh sách file hoặc dùng DB

💡 MẸO LẬP TRÌNH

- Lưu metadata file vào database để dễ quản lý và truy vấn
- Tự động xóa file sau một thời gian nhất định nếu không còn cần thiết
- Tạo API `/files/list` trả về JSON để frontend dễ render
- Tích hợp search/filter/sort cho danh sách file

🛡 BEST PRACTICES

- Lưu trữ metadata file trong database để đồng bộ hóa
- Tự động đồng bộ danh sách file khi upload/download
- Phân quyền chi tiết dựa trên vai trò người dùng
- Tối ưu hiệu suất bằng cache hoặc index trong DB
- Cập nhật UI tự động sau khi upload thành công

🔄 TỔNG KẾT LIÊN KẾT GIỮA CÁC KỸ THUẬT

34. Upload file	35, 36	Là bước đầu tiên trong quy trình upload ↔ download
35. Download file	34, 36	Cần file đã upload thì mới download được
36. Quản lý file	34, 35	Đồng bộ hóa và mở rộng chức năng quản lý file

BÀI HỌC RÚT RA CHO SINH VIÊN

1. **Hiểu rõ từng bước trong quy trình xử lý file:** upload → lưu trữ → download.
2. **Không nên bỏ qua việc kiểm tra đầu vào và phân quyền**, đặc biệt là với ứng dụng sản phẩm thật.
3. **Multer mạnh mẽ nhưng không phải là giải pháp cuối cùng** – cần biết khi nào nên chuyển sang cloud storage hoặc custom handler.
4. **Code sạch + secure + maintainable** luôn là ưu tiên hàng đầu.
5. **Luôn test kỹ các trường hợp ngoại lệ** như file hỏng, tên trùng, truy cập trái phép...

EXERCISE

Multer – Upload và Download file trong Express

Dự án này sẽ giúp bạn củng cố kiến thức về cách xử lý file trên máy chủ Node.js bằng thư viện Multer.

1. TÓM TẮT DỰ ÁN VÀ MỤC TIÊU

- **Tên dự án:** File Sharing Server (Simple)
- **Mục tiêu:** Xây dựng một ứng dụng backend đơn giản cho phép người dùng:
 - Tải lên file (PDF, hình ảnh, tài liệu...) vào server.
 - Xem danh sách các file đã upload.
 - Tải xuống file bất kỳ từ danh sách.
- **Mục tiêu kỹ thuật:** Áp dụng thành thạo các kỹ thuật sau:
 - Setup Express server để sử dụng Multer để upload file vào một thư mục chỉ định.
 - Cấu hình Multer để cho phép download file.
 - Thiết lập Express server để xử lý download file từ một thư mục chỉ định.

2. CHỨC NĂNG CHÍNH

- **Upload file:** Cho phép tải lên một hoặc nhiều file cùng lúc.
- **Danh sách file:** Hiển thị tất cả file đã được upload.
- **Download file:** Cho phép tải file đã upload về máy tính.

3. CÔNG NGHỆ SỬ DỤNG

- **Node.js + Express:** Làm backend.
- **Multer:** Quản lý upload file.
- **path & fs:** Thao tác với hệ thống file.
- **MongoDB (tùy chọn):** Lưu metadata file (nếu bạn muốn mở rộng dự án).
- **Postman / curl:** Kiểm tra API.

4. CẤU TRÚC THƯ MỤC

```
file-sharing-server/
├── public/
│   └── uploads/           # Thư mục lưu trữ file upload
├── routes/
│   ├── upload.route.js   # Route xử lý upload
│   └── download.route.js # Route xử lý download và danh sách file
├── config/
│   └── multer.config.js  # Cấu hình Multer
├── app.js                # Khởi tạo Express app
├── server.js             # Khởi động server
└── package.json
```

5. TÓM TẮT CÁC BƯỚC TRIỂN KHAI

1. Khởi tạo dự án và cài đặt thư viện cần thiết.
2. Cấu hình Multer để upload file vào thư mục `public/uploads`.
3. Tạo route `/upload` để nhận POST request chứa file.
4. Tạo route `/files/list` trả về danh sách file đã upload.
5. Tạo route `/download/:filename` để cho phép tải file về.
6. Viết code đầy đủ cho từng phần.
7. Test bằng Postman hoặc curl.

6. CHI TIẾT TỪNG BƯỚC THỰC HIỆN

Bước 1: Khởi tạo dự án và cài đặt thư viện

Mở terminal hoặc command prompt và thực hiện các lệnh sau:

```
mkdir file-sharing-server
cd file-sharing-server
npm init -y
npm install express multer path fs
```

Tạo thư mục `public/uploads` để lưu trữ file:

```
mkdir -p public/uploads
```

Bước 2: Cấu hình Multer (`config/multer.config.js`)

File này chứa cấu hình cho Multer, bao gồm nơi lưu trữ file, cách đặt tên file và các điều kiện lọc file.

```
const multer = require('multer');
const path = require('path');
const fs = require('fs'); // Import fs để tạo thư mục nếu chưa có

// Định nghĩa đường dẫn lưu trữ file
const uploadDir = 'public/uploads/';

// Tạo thư mục uploads nếu nó chưa tồn tại
if (!fs.existsSync(uploadDir)) {
  fs.mkdirSync(uploadDir, { recursive: true });
}

// Thiết lập nơi lưu trữ file và cách đặt tên file
const storage = multer.diskStorage({
  destination: function (req, file, cb) {
    // Chỉ định thư mục đích để lưu file
    cb(null, uploadDir);
  },
  filename: function (req, file, cb) {
    // Tạo một tên file duy nhất để tránh trùng lặp
```

```

    // Kết hợp timestamp, số ngẫu nhiên và phần mở rộng gốc của file
    const uniqueSuffix = `${Date.now()}-${Math.round(Math.random() * 1E9)}`;
    cb(null, file.fieldname + '-' + uniqueSuffix +
path.extname(file.originalname));
  }
});

// Lọc loại file được phép upload
const fileFilter = (req, file, cb) => {
  // Biểu thức chính quy cho các loại file cho phép (jpeg, jpg, png, pdf)
  const allowedTypes = /jpeg|jpg|png|pdf/;
  // Kiểm tra phần mở rộng file (ví dụ: .jpg, .pdf)
  const extname =
allowedTypes.test(path.extname(file.originalname).toLowerCase());
  // Kiểm tra kiểu MIME của file (ví dụ: image/jpeg, application/pdf)
  const mimetype = allowedTypes.test(file.mimetype);

  if (mimetype && extname) {
    // Nếu cả kiểu MIME và phần mở rộng đều hợp lệ, cho phép upload
    return cb(null, true);
  } else {
    // Ngược lại, từ chối upload và trả về thông báo lỗi
    cb(new Error('Chỉ hỗ trợ file hình ảnh (JPEG, JPG, PNG) và PDF!'));
  }
};

// Khởi tạo Multer với cấu hình đã định nghĩa
const upload = multer({
  storage: storage, // Sử dụng cấu hình lưu trữ đã định nghĩa
  limits: { fileSize: 1024 * 1024 * 5 }, // Giới hạn kích thước file là 5MB (5 *
1024 * 1024 bytes)
  fileFilter: fileFilter // Sử dụng hàm lọc file đã định nghĩa
});

// Export đối tượng upload để có thể sử dụng trong các route khác
module.exports = upload;

```

Bước 3: Route upload file (routes/upload.route.js)

File này định nghĩa route /upload để xử lý các yêu cầu POST để tải file lên server.

```

const express = require('express');
const router = express.Router();
const upload = require('../config/multer.config'); // Import cấu hình Multer

// Định nghĩa route POST cho việc upload file
// upload.single('file') là middleware của Multer để xử lý upload một file

```

```
// 'file' là tên trường (field name) trong form mà client sẽ gửi file lên
router.post('/upload', upload.single('file'), (req, res) => {
  // Kiểm tra xem có file nào được upload hay không
  if (!req.file) {
    // Nếu không có file, trả về lỗi 400 (Bad Request)
    return res.status(400).json({ error: 'Không có file nào được chọn hoặc file không hợp lệ.' });
  }

  // Nếu file được upload thành công, trả về thông báo thành công và tên file
  res.json({
    message: 'File đã được tải lên thành công!',
    filename: req.file.filename // Tên file đã được lưu trên server (do Multer đặt tên)
  });
}, (error, req, res, next) => {
  // Middleware xử lý lỗi của Multer (nếu fileFilter trả về lỗi)
  // Multer sẽ tự động gọi middleware này nếu có lỗi trong quá trình upload (ví dụ: sai loại file)
  if (error instanceof multer.MulterError) {
    // Xử lý các lỗi cụ thể của Multer (ví dụ: FILE_TOO_LARGE)
    return res.status(400).json({ error: error.message });
  } else if (error) {
    // Xử lý các lỗi khác (ví dụ: lỗi từ fileFilter của chúng ta)
    return res.status(400).json({ error: error.message });
  }
  // Chuyển tiếp lỗi nếu không phải lỗi do Multer xử lý
  next(error);
});

module.exports = router;
```

Bước 4: Route lấy danh sách file và download (routes/download.route.js)

File này định nghĩa hai route:

/files/list: Để lấy danh sách tất cả các file đã upload.

/download/:filename: Để tải xuống một file cụ thể dựa trên tên file.

```
const express = require('express');
const router = express.Router();
const fs = require('fs'); // Module để làm việc với hệ thống file
const path = require('path'); // Module để làm việc với đường dẫn file

// Định nghĩa đường dẫn đến thư mục uploads
const uploadsDir = path.join(__dirname, '../public/uploads');
```

```

// Route để lấy danh sách tất cả các file đã upload
router.get('/files/list', (req, res) => {
  // Đọc nội dung thư mục uploads
  fs.readdir(uploadsDir, (err, files) => {
    if (err) {
      // Nếu có lỗi khi đọc thư mục, trả về lỗi 500 (Internal Server Error)
      console.error('Lỗi khi đọc thư mục uploads:', err);
      return res.status(500).json({ error: 'Không thể lấy danh sách file.'
    });
  });
  // Lọc ra các file hợp lệ (không phải thư mục) và trả về dưới dạng JSON
  // Trong trường hợp này, vì uploadsDir chỉ chứa file, chúng ta chỉ cần
  trả về mảng files
  res.json(files);
});

// Route để tải file về
// :filename là một tham số động trong URL, đại diện cho tên file muốn tải
router.get('/download/:filename', (req, res) => {
  const filename = req.params.filename; // Lấy tên file từ tham số URL
  const filePath = path.join(uploadsDir, filename); // Tạo đường dẫn đầy đủ
  đến file

  // Kiểm tra xem file có tồn tại trên server hay không
  if (!fs.existsSync(filePath)) {
    // Nếu file không tồn tại, trả về lỗi 404 (Not Found)
    return res.status(404).send('File không tồn tại trên server.');
```

```

  }

  // Sử dụng res.download() để gửi file về client
  // Hàm này sẽ tự động đặt header Content-Disposition để trình duyệt tải
  file xuống
  res.download(filePath, (err) => {
    if (err) {
      // Xử lý lỗi nếu có vấn đề trong quá trình tải file (ví dụ: file bị
      khóa)
      console.error('Lỗi khi tải file:', err);
      // Kiểm tra lỗi có phải là lỗi đã gửi response rồi không
      if (!res.headersSent) {
        res.status(500).send('Không thể tải file do lỗi máy chủ.');
```

```

  });
});

module.exports = router;
```


Bước 5: Khởi tạo Express App (`app.js`)

File này là nơi khởi tạo ứng dụng Express và tích hợp các route đã tạo. Nó cũng cấu hình Express để phục vụ các file tĩnh (static files).

```
const express = require('express');
const app = express();
const path = require('path'); // Import module path để xử lý đường dẫn

// Import các route đã định nghĩa
const uploadRoute = require('./routes/upload.route');
const downloadRoute = require('./routes/download.route');

// Middleware để phân tích cú pháp JSON từ body của request
// Cho phép Express đọc dữ liệu JSON gửi từ client
app.use(express.json());

// Middleware để phân tích cú pháp URL-encoded data từ body của request
// `extended: true` cho phép phân tích cú pháp các đối tượng lồng nhau
app.use(express.urlencoded({ extended: true }));

// Cấu hình Express để phục vụ các file tĩnh (static files) từ thư mục 'public'
// Điều này cho phép client truy cập trực tiếp các file trong thư mục public
// Ví dụ: http://localhost:3000/uploads/my-image.jpg
app.use(express.static(path.join(__dirname, 'public')));

// Sử dụng các route đã import
// Mọi request đến '/' sẽ được xử lý bởi uploadRoute và downloadRoute
// Điều này có nghĩa là các path như '/upload', '/files/list',
// '/download/:filename' sẽ hoạt động
app.use('/', uploadRoute);
app.use('/', downloadRoute);

// Export đối tượng app để server.js có thể import và khởi động server
module.exports = app;
```

Bước 6: Khởi động server (`server.js`)

File này chịu trách nhiệm khởi động máy chủ Express và lắng nghe các yêu cầu trên một cổng cụ thể.

```
const app = require('./app'); // Import đối tượng Express app từ file app.js
const PORT = process.env.PORT || 3000; // Định nghĩa cổng mà server sẽ lắng nghe
```

```
// Sử dụng process.env.PORT nếu có (dành cho môi trường deploy), nếu không thì
mặc định là 3000

// Lắng nghe các kết nối đến trên cổng đã định nghĩa
app.listen(PORT, () => {
  // Callback function sẽ được gọi khi server bắt đầu lắng nghe
  console.log(`Server đang chạy tại http://localhost:${PORT}`);
  console.log('Để upload file, sử dụng POST request tới /upload với trường
"file".');
  console.log('Để xem danh sách file, truy cập
http://localhost:3000/files/list.');
```

7. HƯỚNG DẪN CHẠY DỰ ÁN

Bước 1: Chạy server

Mở terminal tại thư mục gốc của dự án (file-sharing-server) và chạy lệnh sau:

```
node server.js
```

Server sẽ chạy trên cổng `http://localhost:3000`.

Bước 2: Upload file

Bạn có thể sử dụng Postman hoặc curl để upload file. **Sử dụng curl:**

```
curl -X POST http://localhost:3000/upload -F "file=@/path/to/your/file.jpg"
```

Thay `/path/to/your/file.jpg` bằng đường dẫn thực tế đến file của bạn. Kết quả trả về sẽ là một JSON chứa thông báo thành công và tên file đã upload.

REST API basics: CRUD, test & variable / **Get data**

POST

Params Authorization Headers (11) **Body** Scripts Settings

☐ none ☒ **form-data** ☐ x-www-form-urlencoded ☐ raw ☐ binary ☐ GraphQL

Key	Value	Description
☒ file	File banner.jpg	
Key	Text Value	Description

Body Cookies Headers (7) Test Results (1/1) 200 OK • 49 ms • 339 B

{ } JSON

```

1 {
2   "message": "File đã được tải lên thành công!",
3   "filename": "file-1750320997788-100913625.jpg"
4 }

```

Bước 3: Lấy danh sách file

Sử dụng curl hoặc truy cập trình duyệt:

curl <http://localhost:3000/files/list>

REST API basics: CRUD, test & variable / **Get data**

GET

Params Authorization Headers (11) **Body** Scripts Settings

☐ none ☒ **form-data** ☐ x-www-form-urlencoded ☐ raw ☐ binary ☐ GraphQL

Key	Value
☒ file	File banner.jpg
Key	Text Value

Body Cookies Headers (7) Test Results (1/1)

{ } JSON

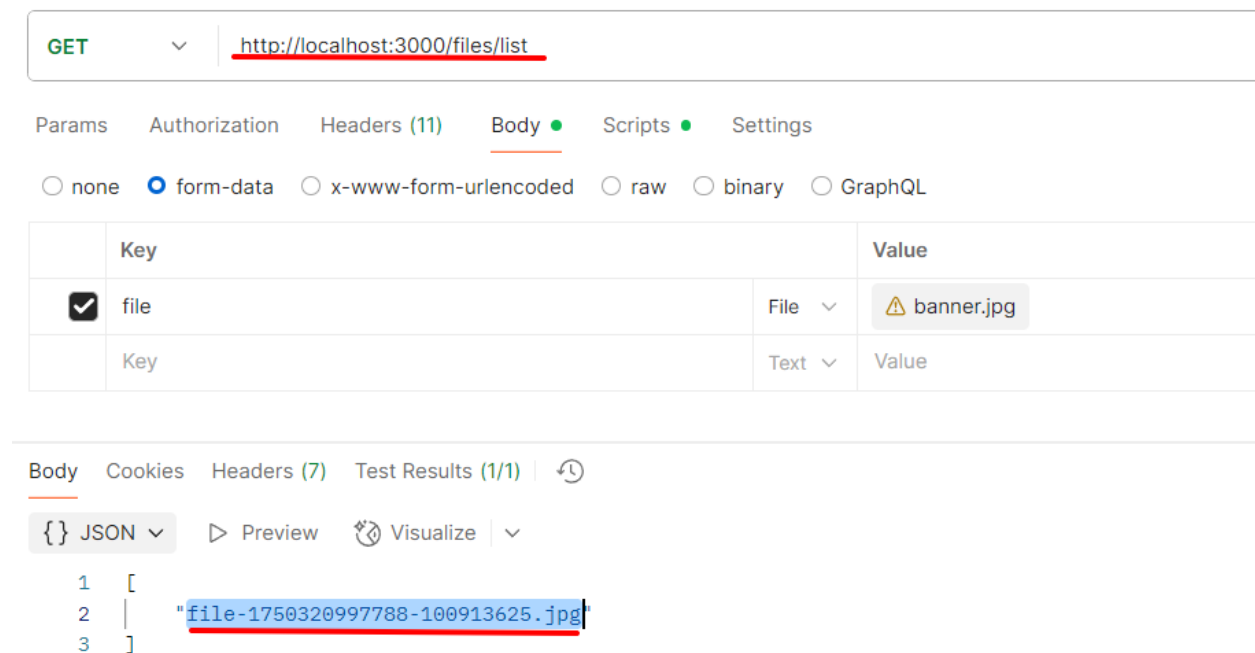
```

1 [
2   "file-1750320997788-100913625.jpg"
3 ]

```

Kết quả sẽ là một mảng JSON chứa tên các file đã upload.

Bước 4: Tải file về



The screenshot displays a REST client interface. At the top, a GET request is configured to `http://localhost:3000/files/list`. Below the request bar, tabs for Params, Authorization, Headers (11), Body, Scripts, and Settings are visible. The Body tab is selected, and the content type is set to 'form-data'. A table below shows a single entry with a checked checkbox, key 'file', and value 'banner.jpg'. The bottom section shows the response body in JSON format, which is an array containing one object with a file name: `[{"file": "file-1750320997788-100913625.jpg"}]`. The file name is highlighted with a red underline.

Key	Value
☒ file	File banner.jpg

Key	Text	Value

Body Cookies Headers (7) Test Results (1/1) | ↻

{ } JSON

```
1 [
2   | "file-1750320997788-100913625.jpg"
3 ]
```

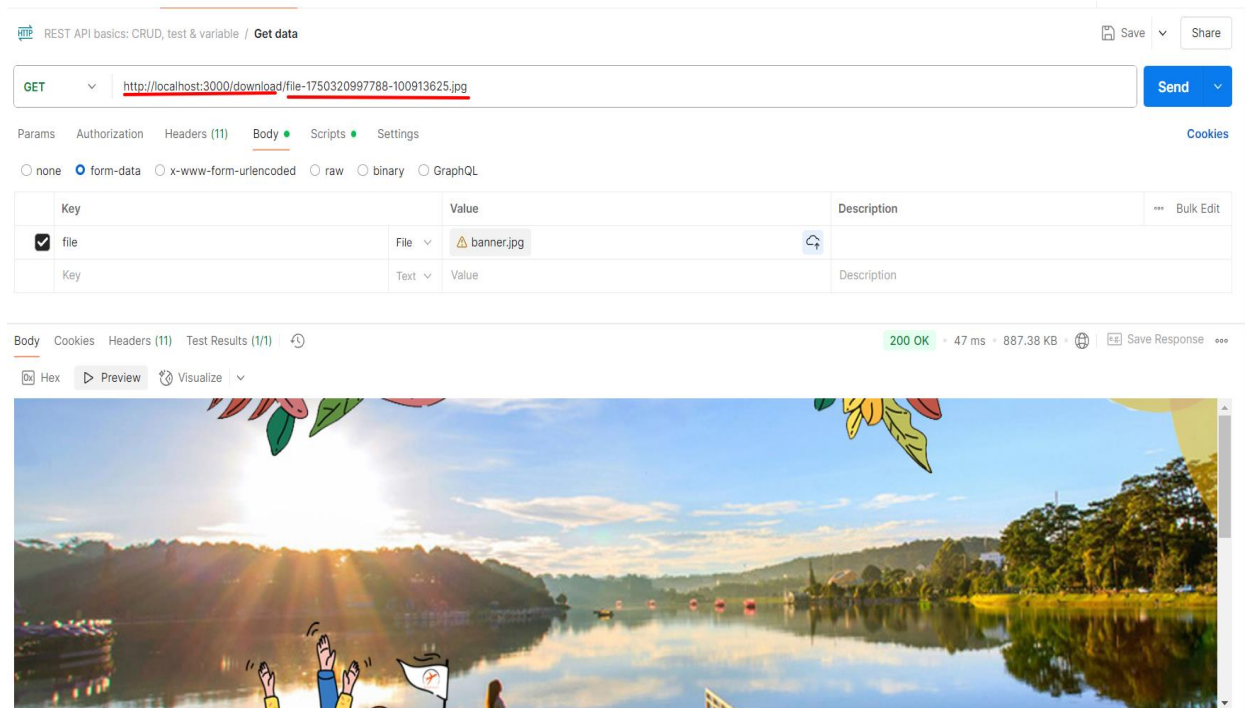
Sử dụng curl hoặc truy cập trình duyệt:

```
curl http://localhost:3000/download/<tên_file> -o <tên_file_luu_tru>
```

Ví dụ:

```
curl http://localhost:3000/download/16788888888-123456789.jpg -o myimage.jpg
```

Thay `<tên_file>` bằng tên file thực tế bạn muốn tải về (tên file bạn nhận được sau khi upload). File sẽ được tải về máy của bạn.



8. KẾT QUẢ ĐẠT ĐƯỢC

Sau khi hoàn thành dự án này, bạn sẽ:

- Thành thạo việc cấu hình Multer để upload file vào thư mục chỉ định.
- Biết cách thiết lập endpoint download file an toàn.
- Có khả năng xây dựng chức năng quản lý file cơ bản.
- Hiểu rõ lỗi thường gặp và cách phòng tránh khi làm việc với file.
- Có tư duy bảo mật và hiệu suất khi làm việc với file.

9. GỢI Ý MỞ RỘNG

- Lưu metadata file vào MongoDB (tên gốc, kích thước, loại, người upload, v.v.).
- Phân quyền upload/download theo vai trò người dùng (role-based access control).
- Tích hợp frontend (React/Vue/Angular) để hiển thị giao diện người dùng thân thiện.
- Hỗ trợ upload nhiều file cùng lúc với `multer().array()` hoặc `multer().fields()`.
- Tự động xóa file sau một thời gian nhất định hoặc khi không còn cần thiết.

10. BÀI HỌC RÚT RA

- Multer là một thư viện mạnh mẽ nhưng cần kết hợp đúng đắn với các module `fs` và `path` của Node.js để xử lý file hiệu quả.
- Luôn kiểm tra đầu vào (input validation), giới hạn kích thước và loại file để đảm bảo an toàn và tránh các lỗ hổng bảo mật.
- Phân tách logic thành các module (file) riêng giúp mã nguồn dễ bảo trì, dễ mở rộng và dễ đọc hơn.

EXERCISE 2

MỞ RỘNG TRONG DỰ ÁN FILE SHARING SERVER

✓ Mục tiêu:

Mở rộng khả năng quản lý file trong ứng dụng bằng cách:

- Lưu trữ metadata (tên, loại, người upload, thời gian...) vào **MongoDB**
- Phân quyền người dùng: chỉ **admin** có thể xóa hoặc xem tất cả file; **user thường** chỉ được upload và download file của mình
- Tích hợp **giao diện frontend với React.js** để hiển thị danh sách file, upload và download trực quan

1. CÔNG NGHỆ SỬ DỤNG THÊM

MongoDB	Cơ sở dữ liệu NoSQL lưu trữ metadata file
Mongoose	Thư viện ODM để thao tác với MongoDB
JWT	Xác thực người dùng và phân quyền
React.js	Giao diện người dùng phía client
Axios	Gọi API từ frontend đến backend

1. Cấu trúc thư mục hoàn chỉnh

text

```
file-sharing-server/
├── public/
│   └── uploads/           # Thư mục lưu trữ file upload
├── routes/
│   ├── auth.route.js      # Route xác thực người dùng
│   └── file.route.js      # Route xử lý file (upload/download/list)
├── controllers/
│   ├── auth.controller.js
│   └── file.controller.js
├── models/
│   ├── File.js            # Mô hình dữ liệu file
│   └── User.js            # Mô hình người dùng
├── middleware/
│   └── auth.middleware.js  # Middleware xác thực
├── config/
│   └── multer.config.js   # Cấu hình upload file
```

		db.js	# Kết nối MongoDB
		jwt.config.js	# Cấu hình JWT
		.env	# Biến môi trường
		app.js	# Cấu hình Express chính
		server.js	# Khởi chạy server
		package.json	

2. Các file code hoàn chỉnh

2.1. File cấu hình MongoDB (`config/db.js`)

javascript

```
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGO_URI, {
      useNewUrlParser: true,
      useUnifiedTopology: true,
    });
    console.log('Kết nối MongoDB thành công');
  } catch (err) {
    console.error('Lỗi kết nối MongoDB:', err.message);
    process.exit(1);
  }
};

module.exports = connectDB;
```

2.2. Model User (`models/User.js`)

javascript

```
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');

const userSchema = new mongoose.Schema({
  username: {
    type: String,
    required: true,
    unique: true,
    trim: true,
    minlength: 3
  },
  password: {
    type: String,
    required: true,
    minlength: 6
  },
  role: {
    type: String,
    default: 'user',
    enum: ['user', 'admin']
  },
  createdAt: {
    type: Date,
    default: Date.now
  }
});
```

```

});

// Hash password trước khi lưu
userSchema.pre('save', async function(next) {
  if (!this.isModified('password')) return next();
  this.password = await bcrypt.hash(this.password, 10);
  next();
});

// Phương thức so sánh password
userSchema.methods.comparePassword = async function(candidatePassword) {
  return await bcrypt.compare(candidatePassword, this.password);
};

module.exports = mongoose.model('User', userSchema);

```

2.3. Model File (models/File.js)

javascript

```

const mongoose = require('mongoose');

const fileSchema = new mongoose.Schema({
  filename: {
    type: String,
    required: true
  },
  originalName: {
    type: String,
    required: true
  },
  size: {
    type: Number,
    required: true
  },
  mimetype: {
    type: String,
    required: true
  },
  uploadedBy: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true
  },
  uploadedAt: {
    type: Date,
    default: Date.now
  }
});

module.exports = mongoose.model('File', fileSchema);

```

2.4. Middleware xác thực (middleware/auth.middleware.js)

javascript

```

const jwt = require('jsonwebtoken');
const User = require('../models/User');

const authMiddleware = async (req, res, next) => {
  try {
    const token = req.header('Authorization')?.replace('Bearer ', '');

```



```

    if (!token) {
      return res.status(401).json({ error: 'Vui lòng đăng nhập để truy cập' });
    }

    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    const user = await User.findById(decoded.id);

    if (!user) {
      return res.status(401).json({ error: 'Người dùng không tồn tại' });
    }

    req.user = user;
    req.token = token;
    next();
  } catch (err) {
    res.status(401).json({ error: 'Phiên đăng nhập không hợp lệ' });
  }
};

const adminOnly = (req, res, next) => {
  if (req.user.role !== 'admin') {
    return res.status(403).json({
      error: 'Truy cập bị từ chối. Chỉ quản trị viên mới được thực hiện thao tác này'
    });
  }
  next();
};

module.exports = { authMiddleware, adminOnly };

```

2.5. Controller Auth (controllers/auth.controller.js)

javascript

```

const jwt = require('jsonwebtoken');
const User = require('../models/User');

exports.register = async (req, res) => {
  try {
    const { username, password, role } = req.body;

    // Kiểm tra username đã tồn tại
    const existingUser = await User.findOne({ username });
    if (existingUser) {
      return res.status(400).json({ error: 'Tên người dùng đã được sử dụng' });
    }

    // Tạo user mới
    const user = new User({ username, password, role });
    await user.save();

    // Tạo token
    const token = jwt.sign({ id: user._id, role: user.role },
      process.env.JWT_SECRET, {
        expiresIn: '1d'
      });

    res.status(201).json({ user, token });
  } catch (err) {

```

```

        res.status(500).json({ error: err.message });
    }
};

exports.login = async (req, res) => {
    try {
        const { username, password } = req.body;
        const user = await User.findOne({ username });

        if (!user || !(await user.comparePassword(password))) {
            return res.status(401).json({ error: 'Tên đăng nhập hoặc mật khẩu không đúng' });
        }

        const token = jwt.sign({ id: user._id, role: user.role },
            process.env.JWT_SECRET, {
                expiresIn: '1d'
            });

        res.json({
            user: {
                id: user._id,
                username: user.username,
                role: user.role
            },
            token
        });
    } catch (err) {
        res.status(500).json({ error: err.message });
    }
};

exports.getMe = async (req, res) => {
    res.json({
        user: {
            id: req.user._id,
            username: req.user.username,
            role: req.user.role
        }
    });
};

```

2.6. Controller File (controllers/file.controller.js)

javascript

```

const fs = require('fs');
const path = require('path');
const File = require('../models/File');
const User = require('../models/User');

// Upload file
exports.uploadFile = async (req, res) => {
    try {
        if (!req.file) {
            return res.status(400).json({ error: 'Vui lòng chọn file để upload' });
        }

        const file = new File({
            filename: req.file.filename,
            originalName: req.file.originalname,

```

```

        size: req.file.size,
        mimetype: req.file.mimetype,
        uploadedBy: req.user._id
    });

    await file.save();

    res.status(201).json({
        message: 'Upload file thành công',
        file: {
            id: file._id,
            originalName: file.originalName,
            size: file.size,
            uploadedAt: file.uploadedAt
        }
    });
} catch (err) {
    // Xóa file đã upload nếu có lỗi
    if (req.file) {
        fs.unlinkSync(req.file.path);
    }
    res.status(500).json({ error: err.message });
}
};

// Danh sách file
exports.listFiles = async (req, res) => {
    try {
        let files;
        if (req.user.role === 'admin') {
            files = await File.find().populate('uploadedBy', 'username');
        } else {
            files = await File.find({ uploadedBy: req.user._id
        }).populate('uploadedBy', 'username');
        }

        res.json(files);
    } catch (err) {
        res.status(500).json({ error: err.message });
    }
};

// Download file
exports.downloadFile = async (req, res) => {
    try {
        const file = await File.findOne({ filename: req.params.filename });
        if (!file) {
            return res.status(404).json({ error: 'File không tồn tại' });
        }

        // Kiểm tra quyền truy cập
        if (req.user.role !== 'admin' &&
            !file.uploadedBy.equals(req.user._id)) {
            return res.status(403).json({ error: 'Bạn không có quyền truy cập
file này' });
        }

        const filePath = path.join(__dirname, '../../public/uploads',
file.filename);
        if (!fs.existsSync(filePath)) {
            return res.status(404).json({ error: 'File không tồn tại trên
server' });
        }
    }
};

```

```

        res.download(filePath, file.originalName);
    } catch (err) {
        res.status(500).json({ error: err.message });
    }
};

// Xóa file (chỉ admin)
exports.deleteFile = async (req, res) => {
    try {
        const file = await File.findById(req.params.id);
        if (!file) {
            return res.status(404).json({ error: 'File không tồn tại' });
        }

        const filePath = path.join(__dirname, '../public/uploads',
file.filename);

        // Xóa file từ hệ thống file
        if (fs.existsSync(filePath)) {
            fs.unlinkSync(filePath);
        }

        // Xóa record từ database
        await file.remove();

        res.json({ message: 'Xóa file thành công' });
    } catch (err) {
        res.status(500).json({ error: err.message });
    }
};

```

2.7. Cấu hình Multer (config/multer.config.js)

javascript

```

const multer = require('multer');
const path = require('path');

// Cấu hình lưu trữ file
const storage = multer.diskStorage({
    destination: (req, file, cb) => {
        cb(null, path.join(__dirname, '../public/uploads'));
    },
    filename: (req, file, cb) => {
        const uniqueSuffix = Date.now() + '-' + Math.round(Math.random() *
1E9);
        cb(null, uniqueSuffix + path.extname(file.originalname));
    }
});

// Kiểm tra loại file
const fileFilter = (req, file, cb) => {
    const allowedTypes = ['image/jpeg', 'image/png', 'application/pdf',
'text/plain'];
    if (allowedTypes.includes(file.mimetype)) {
        cb(null, true);
    } else {
        cb(new Error('Loại file không được hỗ trợ'), false);
    }
};

```

```

const upload = multer({
  storage,
  fileFilter,
  limits: {
    fileSize: 1024 * 1024 * 5 // Giới hạn 5MB
  }
});

module.exports = upload;

```

2.8. Route Auth (`routes/auth.route.js`)

javascript

```

const express = require('express');
const router = express.Router();
const authController = require('../controllers/auth.controller');

// Đăng ký tài khoản
router.post('/register', authController.register);

// Đăng nhập
router.post('/login', authController.login);

// Lấy thông tin user hiện tại
router.get('/me', authController.getMe);

module.exports = router;

```

2.9. Route File (`routes/file.route.js`)

javascript

```

const express = require('express');
const router = express.Router();
const fileController = require('../controllers/file.controller');
const { authMiddleware, adminOnly } =
  require('../middleware/auth.middleware');
const upload = require('../config/multer.config');

// Upload file (yêu cầu đăng nhập)
router.post('/upload', authMiddleware, upload.single('file'),
  fileController.uploadFile);

// Danh sách file
router.get('/list', authMiddleware, fileController.listFiles);

// Download file
router.get('/download/:filename', authMiddleware,
  fileController.downloadFile);

// Xóa file (chỉ admin)
router.delete('/:id', authMiddleware, adminOnly,
  fileController.deleteFile);

module.exports = router;

```

2.10. File app.js

javascript

```

const express = require('express');
const cors = require('cors');
const helmet = require('helmet');
const morgan = require('morgan');
const path = require('path');

const authRoutes = require('./routes/auth.route');
const fileRoutes = require('./routes/file.route');

const app = express();

// Middleware
app.use(cors());
app.use(helmet());
app.use(morgan('dev'));
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

// Static files
app.use('/uploads', express.static(path.join(__dirname, 'public/uploads')));

// Routes
app.use('/api/auth', authRoutes);
app.use('/api/files', fileRoutes);

// Error handling
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({ error: 'Đã xảy ra lỗi trên server' });
});

module.exports = app;

```

2.11. File server.js

javascript

```

const app = require('./app');
const connectDB = require('./config/db');
require('dotenv').config();

const PORT = process.env.PORT || 5000;

// Kết nối database
connectDB();

// Khởi động server
app.listen(PORT, () => {
  console.log(`Server đang chạy trên port ${PORT}`);
});

```

3. HƯỚNG DẪN TRIỂN KHAI TỪNG BƯỚC

3.1. Chuẩn bị môi trường

1. Cài đặt Node.js và MongoDB

- Cài đặt Node.js phiên bản LTS từ <https://nodejs.org>
- Cài đặt MongoDB Community Edition từ <https://www.mongodb.com/try/download/community>

2. Tạo thư mục dự án

bash

```
mkdir file-sharing-server
cd file-sharing-server
npm init -y
```

• Cài đặt các dependencies

bash

```
npm install express mongoose multer jsonwebtoken bcryptjs dotenv cors
helmet morgan
```

• Tạo file .env

text

```
MONGO_URI=mongodb://localhost:27017/file_sharing
JWT_SECRET=your_jwt_secret_key
PORT=5000
```

3.2. Khởi chạy backend

1. Tạo thư mục uploads

bash

```
mkdir -p public/uploads
```

• Khởi động MongoDB

Đảm bảo MongoDB service đang chạy

• Khởi động server

bash

```
node server.js
```

3.3. Triển khai Frontend với React

1. Tạo ứng dụng React

bash

```
npx create-react-app frontend
cd frontend
npm install axios react-router-dom
```

- **Cấu trúc thư mục frontend**

text

- frontend/
 - public/
 - src/
 - components/
 - Auth/
 - Login.jsx
 - Register.jsx
 - File/
 - FileList.jsx
 - UploadForm.jsx
 - Layout/
 - Navbar.jsx
 - PrivateRoute.jsx
 - pages/
 - Dashboard.jsx
 - Home.jsx
 - Admin.jsx
 - App.jsx
 - main.jsx
 - utils/
 - api.js
 - package.json

- **File api.js (frontend/src/utils/api.js)**

javascript

```
import axios from 'axios';

const API_URL = 'http://localhost:5000/api';

const api = axios.create({
  baseURL: API_URL,
  headers: {
    'Content-Type': 'application/json'
  }
});

// Thêm interceptor để tự động thêm token vào header
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
}, (error) => {
```



```

    return Promise.reject(error);
  });

  export const register = (userData) => api.post('/auth/register',
userData);
  export const login = (userData) => api.post('/auth/login', userData);
  export const getMe = () => api.get('/auth/me');

  export const uploadFile = (file) => {
    const formData = new FormData();
    formData.append('file', file);

    return api.post('/files/upload', formData, {
      headers: {
        'Content-Type': 'multipart/form-data'
      }
    });
  };

  export const listFiles = () => api.get('/files/list');
  export const downloadFile = (filename) =>
api.get(`/files/download/${filename}`, { responseType: 'blob' });
  export const deleteFile = (id) => api.delete(`/files/${id}`);

  export default api;

```

- **Component UploadForm (frontend/src/components/File/UploadForm.jsx)**

jsx

```

import { useState } from 'react';
import { uploadFile } from '../../utils/api';

function UploadForm({ onUploadSuccess }) {
  const [file, setFile] = useState(null);
  const [error, setError] = useState('');
  const [isUploading, setIsUploading] = useState(false);

  const handleFileChange = (e) => {
    setFile(e.target.files[0]);
  };

  const handleSubmit = async (e) => {
    e.preventDefault();
    if (!file) {
      setError('Vui lòng chọn file');
      return;
    }

    try {
      setIsUploading(true);
      setError('');
      await uploadFile(file);
      onUploadSuccess();
      alert('Upload file thành công');
    } catch (err) {
      setError(err.response?.data?.error || 'Upload file thất bại');
    } finally {
      setIsUploading(false);
    }
  };

  return (

```

```

    <div className="upload-form">
      <h3>Upload File</h3>
      {error && <div className="error">{error}</div>}
      <form onSubmit={handleSubmit}>
        <input type="file" onChange={handleFileChange} />
        <button type="submit" disabled={isUploading}>
          {isUploading ? 'Đang upload...' : 'Upload'}
        </button>
      </form>
    </div>
  );
}

export default UploadForm;

```

• Component FileList (frontend/src/components/File/FileList.jsx)

jsx

```

import { useState, useEffect } from 'react';
import { listFiles, downloadFile, deleteFile } from '../../utils/api';

function FileList({ isAdmin }) {
  const [files, setFiles] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState('');

  const fetchFiles = async () => {
    try {
      setLoading(true);
      const { data } = await listFiles();
      setFiles(data);
    } catch (err) {
      setError(err.response?.data?.error || 'Lỗi khi tải danh sách
file');
    } finally {
      setLoading(false);
    }
  };

  useEffect(() => {
    fetchFiles();
  }, []);

  const handleDownload = async (filename, originalName) => {
    try {
      const response = await downloadFile(filename);
      const url = window.URL.createObjectURL(new Blob([response.data]));
      const link = document.createElement('a');
      link.href = url;
      link.setAttribute('download', originalName);
      document.body.appendChild(link);
      link.click();
    } catch (err) {
      alert(err.response?.data?.error || 'Lỗi khi tải file');
    }
  };

  const handleDelete = async (id) => {
    if (!window.confirm('Bạn có chắc chắn muốn xóa file này?')) return;

    try {
      await deleteFile(id);
    }
  };
}

```

```

        fetchFiles();
    } catch (err) {
        alert(err.response?.data?.error || 'Lỗi khi xóa file');
    }
};

if (loading) return <div>Đang tải...</div>;
if (error) return <div className="error">{error}</div>;

return (
    <div className="file-list">
        <h3>Danh sách file</h3>
        <table>
            <thead>
                <tr>
                    <th>Tên file</th>
                    <th>Người upload</th>
                    <th>Ngày upload</th>
                    <th>Hành động</th>
                </tr>
            </thead>
            <tbody>
                {files.map((file) => (
                    <tr key={file._id}>
                        <td>{file.originalName}</td>
                        <td>{file.uploadedBy?.username}</td>
                        <td>{new Date(file.uploadedAt).toLocaleString()}</td>
                        <td>
                            <button onClick={() => handleDownload(file.filename,
file.originalName)}>
                                Tải về
                            </button>
                            {isAdmin && (
                                <button onClick={() => handleDelete(file._id)}>
                                    Xóa
                                </button>
                            )}
                        </td>
                    </tr>
                ))}
            </tbody>
        </table>
    </div>
);
}

export default FileList;

```

4. Hướng dẫn sử dụng

4.1. Đăng ký tài khoản

1. Gửi POST request đến `/api/auth/register` với body:

json

- {


```

            "username": "user1",
            "password": "password123",
            "role": "user" // hoặc "admin"
        }

```

- Server sẽ trả về thông tin user và token:

json

```
{
  "user": {
    "_id": "...",
    "username": "user1",
    "role": "user",
    "createdAt": "..."
  },
  "token": "jwt_token"
}
```

4.2. Đăng nhập

1. Gửi POST request đến /api/auth/login với body:

json

```
{
  "username": "user1",
  "password": "password123"
}
```

- Server trả về token nếu thành công:

json

```
{
  "user": {
    "id": "...",
    "username": "user1",
    "role": "user"
  },
  "token": "jwt_token"
}
```

4.3. Upload file

1. Gửi POST request đến /api/files/upload với header:

text

```
Authorization: Bearer <token>
Content-Type: multipart/form-data
```

- Đính kèm file trong form-data với key là file
- Server trả về thông tin file nếu thành công:

json

```
{
  "message": "Upload file thành công",
  "file": {
    "id": "...",

```

```

        "originalName": "example.pdf",
        "size": 12345,
        "uploadedAt": "..."
    }
}

```

4.4. Xem danh sách file

1. Gửi GET request đến `/api/files/list` với header:

text

```
Authorization: Bearer <token>
```

- Server trả về danh sách file:

json

```

[
  {
    "_id": "...",
    "filename": "12345-example.pdf",
    "originalName": "example.pdf",
    "size": 12345,
    "uploadedBy": {
      "_id": "...",
      "username": "user1"
    },
    "uploadedAt": "..."
  }
]

```

4.5. Download file

1. Gửi GET request đến `/api/files/download/:filename` với header:

text

```
Authorization: Bearer <token>
```

1. Server sẽ trả về file để download

4.6. Xóa file (chỉ admin)

1. Gửi DELETE request đến `/api/files/:id` với header:

text

```
Authorization: Bearer <token>
```

- Server trả về thông báo thành công:

json

```

{
  "message": "Xóa file thành công"
}

```

}

5. Bài tập mở rộng cho sinh viên

1. **Thêm chức năng phân trang** cho danh sách file
2. **Tích hợp tìm kiếm** file theo tên hoặc loại file
3. **Thêm chức năng chia sẻ file** giữa các user
4. **Tích hợp thông báo real-time** với Socket.io khi có file mới được upload
5. **Thêm chức năng preview** cho các file hình ảnh và PDF trực tiếp trên trình duyệt
6. **Tích hợp xác thực 2 bước** cho tài khoản admin
7. **Thêm API documentation** sử dụng Swagger